Notes Week 25

a. Intershell abundances

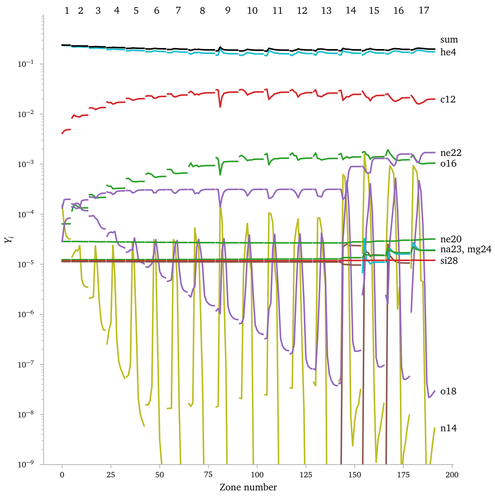
Using the provided script I was able to, quite easily access the intershell abundances.

However, the structure is not immediately very clear to me. It seems like there are multiple abundances for every.

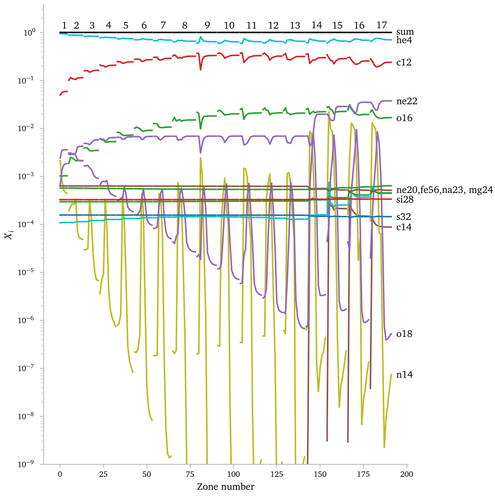
Then I also need to figure out what the actual “abundances” mean.

A quick test is to simply sum the “abundances” of a TP and see what the sum is equal to.

Here, I show the “abundances” for the $M_{\mathrm{TPAGB},i} = 1.5\ M_\odot$ star, with $z = 0.007$ and $M_{\mathrm{min}}/M_\odot = 2 \times 10^{-3}$, which is what Karakas & Lugaro (2016) indicates as the “standard” choice.

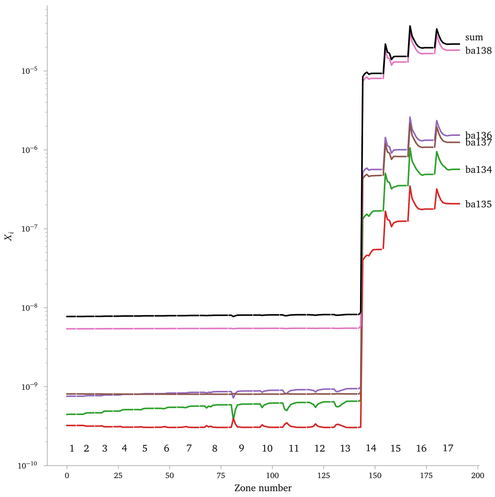


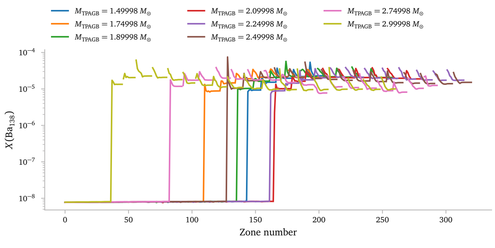
Clearly, this is *not* a mass ratio, since this would sum to 1. Now, if we multiply it by the atomic mass, we get the following picture.



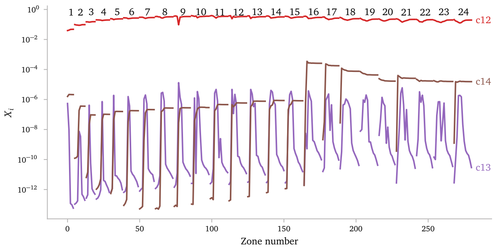
Now we see that the elements do sum to 1. Therefore, the abundances are given in terms of number ratios. I find mass abundances to be a bit more intuitive so I think I will work with those from now on.

Let me look at some abundances for barium:



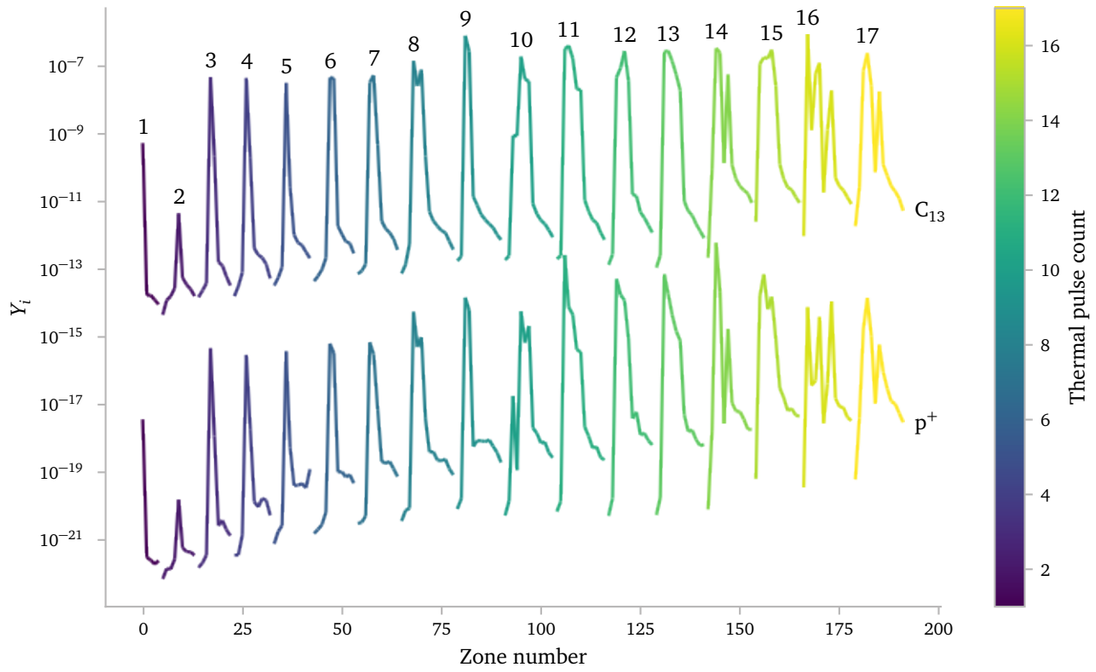


Here I show the carbon abundances for the $M = 2.0\ M_\odot$ model. Here we



see that the C_{13} abundances have very clear pulse-like shape, just like the O_{18} and N_{14} abundances for example.

Next, I plot the intershell proton number ratio and the C_{13} number ratio. They follow a very similar pattern. I know TPAGB stars get C_{13} pockets, so I wonder if the different abundances per pulse have to do with this?



Below, I compare the total metals mass fraction for stars of different masses. I also compare it with the intershell metal abundances of a *MESA* model[1]. It is important to note that the MESA model only contains profiles during the thermal pulse. This seems to most closely match the properties of the intershell abundances from Zara. It is also important to realise that the horizontal axes has little meaning here and should *not* be used to make real comparisons. For example, the *MESA* model has much fewer models during the first thermal pulses than the last, which makes the last pulses seem much more important.

1 see next section.

The overal shape and metal fractions align well I would say.

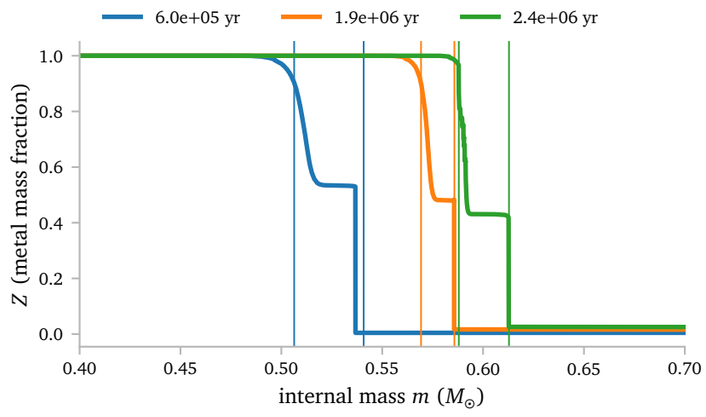
b. Testing MESA dredge-up

To validate the abundance post-processing procedure that I am to use, we can look at elements for which we *know* the abundances both in the intershell region, *and* in the envelope. We can then *just* use the intershell abundances, the mass of dredge-up and the mass of the convective envelope to predict the envelope abundances.

I have previously simulated a single star ($z = 0.00453$ *and* $M_{\mathrm{TPAGB,i}} = 2\,M_\odot$) with *a lot* of profiles throughout the complete evolution.

This star does not have individual abundances for isotopes, *but* it does have the hydrogen, helium and metal mass fractions. We can use this metal mass fraction. It should not matter if we are looking at a single isotope or if we are looking at the combination of a bunch of isotopes.

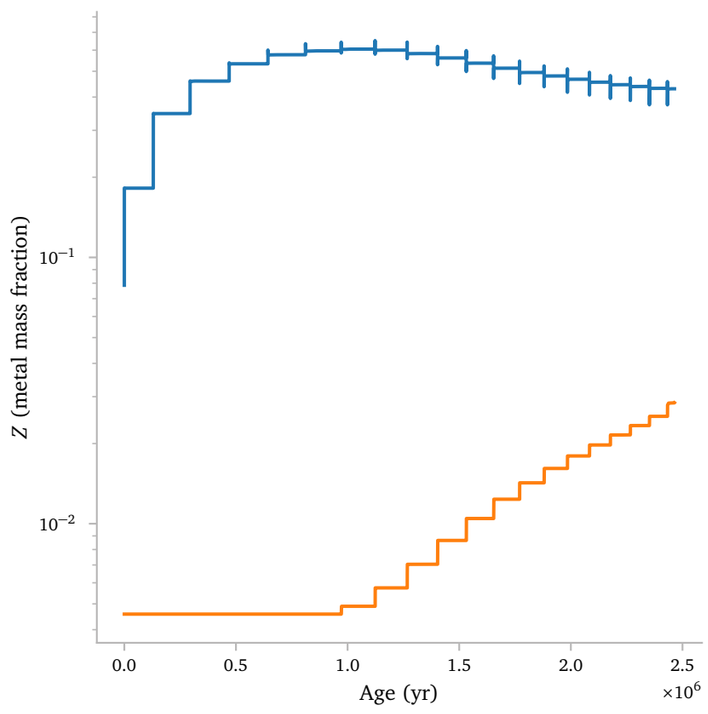
Let me show the metal mass fraction for a couple of profiles. This interactive plot however, shows the mass fraction of metals as a function of time.



The vertical lines denote the border of the CO-core and the helium core. The intershell region lies between these two lines. The border between the CO-core and the intershell region is not very sharp. This is however not a big problem because the TDU does not get close to this border at all. This can clearly be seen in the interactive figure.

After the gradual border between the CO-core and the intershell region, we see that the metal mass fraction is roughly constant. We can use an average here to determine the metal mass fraction in the intershell region as a function of time.

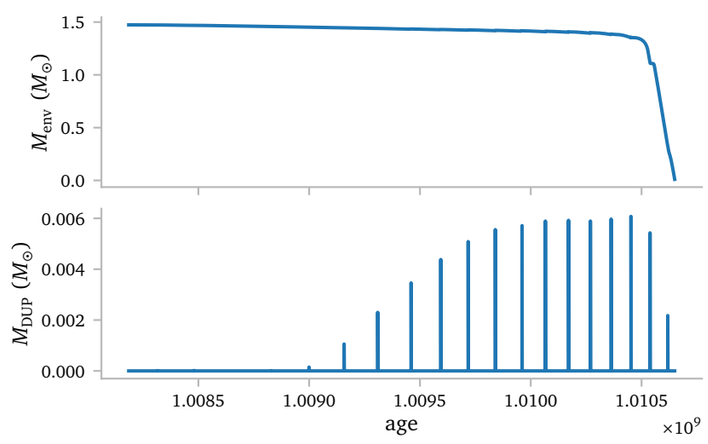
Below I plot this, as well as a similarly retrieved envelope abundance:



The intershell abundances actually have some slight pulsation during thermal pulses. I do not think this is noise and I expect this to be a real effect.

We can also clearly see that the envelope does not get enriched during the first thermal pulses. This is in line with what we expect, where there is nearly no TDU during the earlier pulses.

From the `history` data, we know M_{DUP} and M_{env}. For the sake of completeness, let me plot these as well.



I am approximating the dredge-up as happening instantaneously. This means we are seeing a bunch of δ-functions here.

Since my approximation uses instantaneous dredge-up, I then use the following equation:

$$\begin{aligned}\Delta Z_{\mathrm{env}} &= \frac{M_{Z,\mathrm{env}} + \Delta M_{Z,\mathrm{env}}}{M_{\mathrm{env}} + \Delta M_{\mathrm{env}}} - \frac{M_{Z,\mathrm{env}}}{M_{\mathrm{env}}} \\ &= \frac{M_{Z,\mathrm{env}} + M_{\mathrm{DUP}} \cdot Z_{\mathrm{is}}}{M_{\mathrm{env}} + M_{\mathrm{DUP}}} - \frac{M_{Z,\mathrm{env}}}{M_{\mathrm{env}}}\end{aligned}$$

or, for implementation in code:

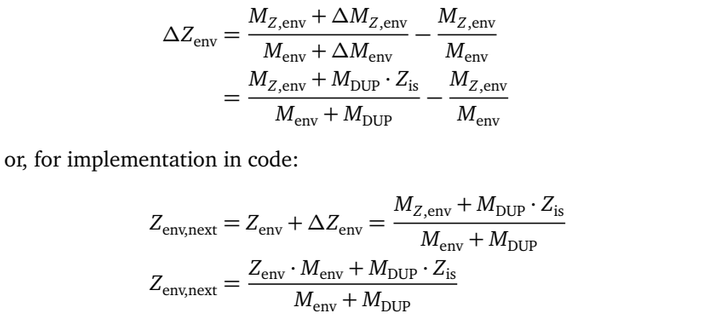
$$\begin{aligned}Z_{\mathrm{env,next}} &= Z_{\mathrm{env}} + \Delta Z_{\mathrm{env}} = \frac{M_{Z,\mathrm{env}} + M_{\mathrm{DUP}} \cdot Z_{\mathrm{is}}}{M_{\mathrm{env}} + M_{\mathrm{DUP}}} \\ Z_{\mathrm{env,next}} &= \frac{Z_{\mathrm{env}} \cdot M_{\mathrm{env}} + M_{\mathrm{DUP}} \cdot Z_{\mathrm{is}}}{M_{\mathrm{env}} + M_{\mathrm{DUP}}}\end{aligned}$$

Below we see the results of this. Similarly to how I computed the envelope

Here I plot the actual mass fraction of metals as a function of time and the post-processed mass fraction of metals.

The envelope abundances are *quite* close but not identical. It seems like we *slightly* overestimate the effects of dredge-up.

abundances, we can also compute M_{DUP} using the intershell and envelope abundances and compare this mass with the dredge-up mass that I find using just the history data.



Here we see nearly the same thing. We slightly overestimate the amount of dredge-up that occurs.

c. Reviving MESA models I

c.1 Introduction

Last week I showed this image: Clearly my fix works pretty well. Now only about 5% of the models fail due to convergence problems.

c.2 Method

My basic idea consists of 3 parts:

1. **Ensure small envelope changes by limiting the maximum change in ΔM_{env}.** By continuously decreasing the maximum change in stellar mass per timestep depending on the envelope mass, I can assure that at no point, the changes between timesteps are too large.
2. **Help the star smooth over convergence problems changing the mesh resolution.** By changing the mesh resolution during the interpulse when the mass loss rate is large (a larger value meaning *less* zones), the star can from experience often resume its evolution without convergence problems.
3. **Restart the star with a different mesh resolution after convergence problems.** By changing the mesh resolution after convergence problems, the star might be able to continue past a previously problematic point. We should then slowly return to the original resolution afterwards.

The first two points I have briefly discussed last week as well, but I will repeat them here for completeness.

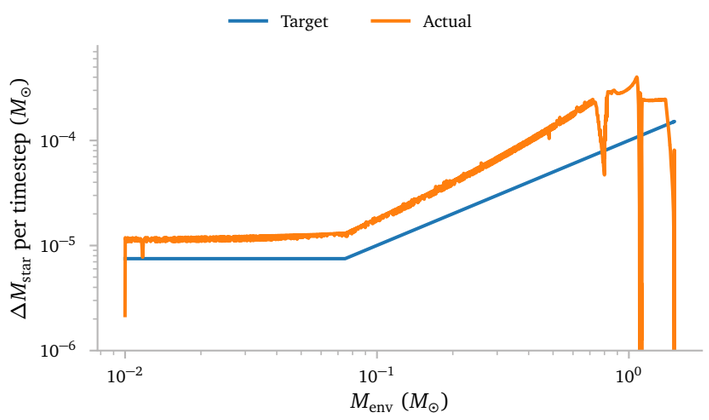
c.2.1 Limiting changes in mass

This is pretty straightforward. I use the relation for the limit on the change in mass:

$$\log(\Delta M_{\text{lim}}) = \begin{cases} 7.5 \cdot 10^{-6} & \text{, if} \quad M_{\text{env}} < 0.075\ M_\odot, \\ 10^{-4} \cdot M_{\text{env}} \cdot 10^{-6} & \text{, else} \end{cases}.$$

This is the implementation in the star extra code:

```
subroutine update_max_dm(s, ierr)
    integer, intent(out) :: ierr
    type (star_info), pointer :: s

    real(dp) :: min_dm = 7.5d-6
    real(dp) :: max_dm = 1.0d-3
    real(dp) :: envelope_mass

    envelope_mass = s% star_mass - s% he_core_mass

    s% delta_lg_star_mass_limit = min( max(envelope_mass*1d-4, min_dm), max_dm)

end subroutine update_max_dm
```



I am not sure why the star is not actually respecting the limitation but is always slightly above it, but in any case, it does seem to following the relation and the change in mass is never too large so I am content with this.

c.2.2 Mesh refinement

The mesh refinement code is pretty simple:

```
subroutine resolve_mesh_during_mass_loss(s, ierr)
    ! Increase mesh_delta_coeff during strong envelope stripping
    ! outside of thermal pulses, to reduce the number of cells.
    !
    ! call in extras_finish_step
    type (star_info), pointer :: s
    integer, intent(out) :: ierr

    real(dp), parameter :: lg_mdot_limit = -5.5d0
    real(dp) :: target_mesh_delta_coeff = 1.0d0

    ierr = 0


    if (s%lxtra(lx_in_LHe_peak)) then
        target_mesh_delta_coeff = 1.0d0

    else if (log10(abs(s%star_mdot)) > lg_mdot_limit) then
        target_mesh_delta_coeff = 2.0d0

    else
        target_mesh_delta_coeff = 1.0d0
    end if

    if (s%mesh_delta_coeff < target_mesh_delta_coeff) then
        s%mesh_delta_coeff = min( &
            s%mesh_delta_coeff + 0.01d0, &
            target_mesh_delta_coeff)
            write(*,*) 'strong mass loss: increasing mesh_delta_coeff = ', &
                s%mesh_delta_coeff

    else if (s%mesh_delta_coeff > target_mesh_delta_coeff) then
        s%mesh_delta_coeff = max( &
            s%mesh_delta_coeff - 0.01d0, &
            target_mesh_delta_coeff)
            write(*,*) 'TP: decreasing mesh_delta_coeff = ', &
                s%mesh_delta_coeff
    end if

end subroutine resolve_mesh_during_mass_loss
```

c.2.3 Restarting simulations

I wrote a python script to automatically restart *MESA* simulations when they encounter difficult convergence regions. The simulation restarts with a different mesh resolution, which *forces* the star to recompute the envelope structure. This can often help the star evolve past its original problematic region.

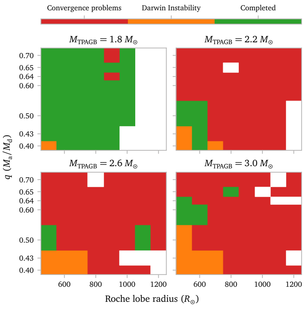
Let me start with the core loop:

```
# all models will need to first be cleaned, then compiled.
# without this, we cannot start or restart any simulation.

print("cleaning dir")
subprocess.run(["./clean"])
print("making binaries")
subprocess.run(["./mk"])

restarts = 1

# first i test if the model has already previously been
# simulated. if not, we need to start from the beginning,
# otherwise we can skip straight to the restart loop.

try:
    # only models that have succesfully been simulated
    # will have an uncorrupted history.data file.

    history = mr.MesaData(f"LOGS/history.data")
except FileNotFoundError:
    print("running from scratch because data is missing")
    subprocess.run([f"./rn"])


prev_start_model = None

# the allow each model to restart 10 times.
# if it has not completed by then, it is stuck.

while restarts <= 10 and not check_finished():

    # for every restart, we need to obtain a starting photo,
    # a starting mesh_delta_coeff and a starting model number

    photo_int, mesh_delta_coeff, start_model = get_restart(prev_start_model)

    # we then update the inlist of the simulation
    # with the new mesh_delta_coeff
    update_inlist(mesh_delta_coeff)

    # then we run the simulation.
    print(photo_int)
    subprocess.run([f"./re", photo_int])

    # if the simulation is finished, we break from the loop.
    if check_finished():
        break
    prev_start_model = start_model

    # else, we increment the restarts counter.
    restarts += 1
```

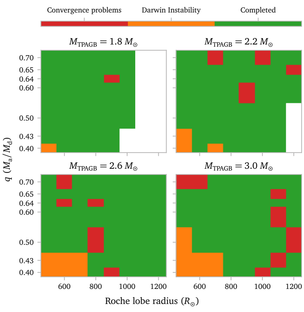
I put the complete code in the appendix of this document.

c.3 Grid Results

To start, I want to compare to look at the final fate of the models. Again, let me first compare it with the results from last week:

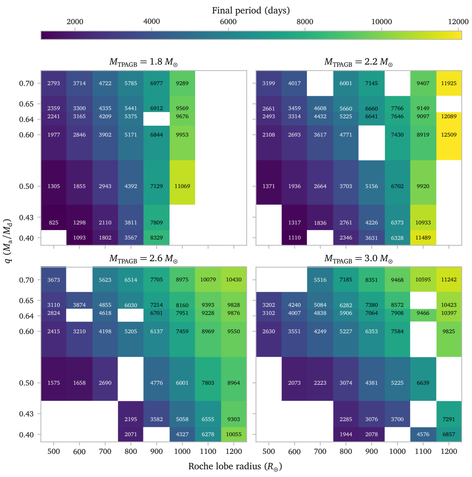


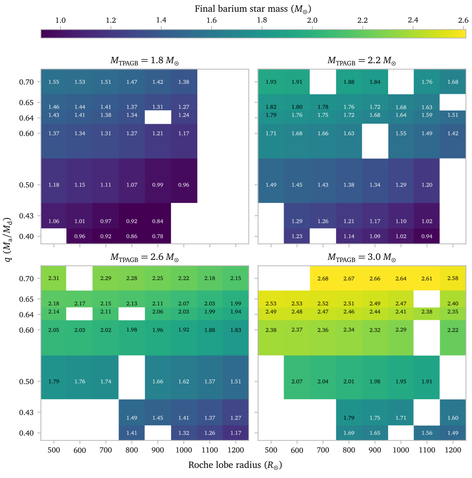
This is the grid as it was last week. We see that for $M = 2.2\ M_\odot$, 2.6 $M_\odot$ and 3.0 $M_\odot$, nearly all models crashed.



This is the grid now. At this point, there are only a few models in every grid that did not simulate properly.

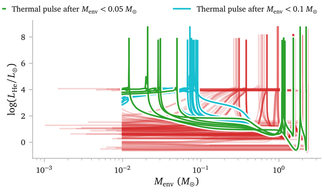
Below, I plot the final period of the binaries.



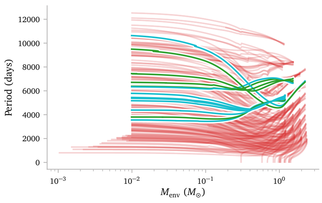


While inspecting the progress of the models on the cluster, I noticed that there were a few models that seemed to experience a thermal pulse very close to the end of the simulation [2]. This made me want to look into this phenomenon.

2 read, when they only have very little envelope left.

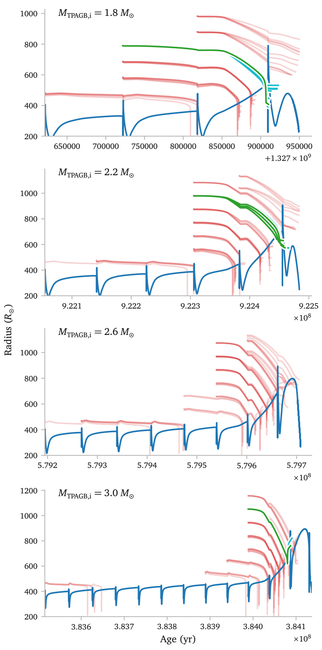


Here I show the helium luminosity as a function of envelope mass. The spikes correspond to thermal pulses and it is clear that there are some models that experience very late thermal pulses (with very late thermal pulse, I do not mean an actual VLT which would happen during the WD cooling track, but very late in relation to the simulation duration). I categorize them in 3 groups which I will use in the next figures.



To start, I wanted to investigate the effect of these late thermal pulses on the binary period.

I guess unsurprisingly, these late pulses have little to no effect. The mass lost can only be very little anyway.



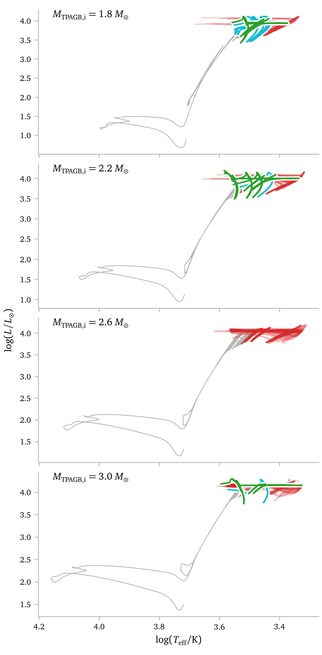
Here, I plot the Roche lobe radius of the binaries and the Radius of the reference star.

The green and blue lines undergo RLOF just before the single star would undergo a thermal pulse.

I think the initial Roche lobe radius could therefore be a good predictor for LTPs (this time I mean *actual* late thermal pulses).

For example, looking at this plot, I think the $R_{\rm RL,i} = 1000\ R_\odot$ models could probably undergo a late thermal pulse if I continued the simulation, just because the stars would continue evolving on the post-AGB track.

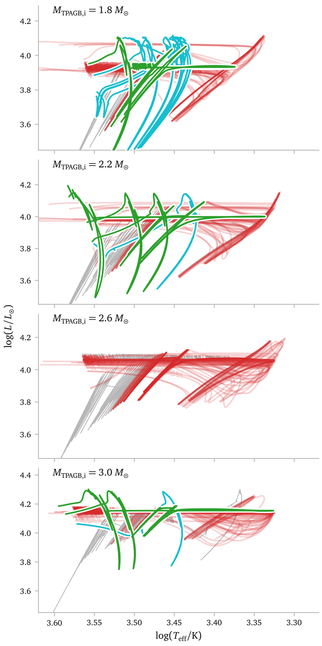
This plot nicely shows how it is *much* more difficult to make the binary interact during a certain thermal pulse *before* the strong winds start as opposed to when the strong winds have started.



Here I show the *full* HR diagrams for the simulation. The gray line is the single star, which continues into the colored lines for the different binary simulations. The colors are the same as in the figures above, meaning the green models experience the late thermal pulses.

Clearly, when viewed in the full HR diagram, the thermal pulses do not appear to happen in a very different place than the earlier pulses. This is in contrast to the thermal pulses that happen during for example the white dwarf cooling track of late during the post-age.

To make the late pulses more visible, let me zoom in on only the TPAGB-region.



d. Reviving MESA models II

As I have shown in the previous section, my code to "revive" *MESA* models works very well. However, there are still a handful of models that fail to converge. Looking at the output, we see that the models get stuck in a loop where they keep trying the same two starting photos in a row.

```
cleaning dir
making binaries
11800, not enough models between
11750, possible, testing retries
retries succesful
x750
18250, not enough models between
18200, possible, testing retries
retries succesful
x200
18200, not enough models between
18150, possible, testing retries
retries succesful
x150
18250, not enough models between
18200, possible, testing retries
retries succesful
x200
18300, not enough models between
18250, possible, testing retries
retries failed
18200, want to select the same starting model as last time. selecting earlier model.
18150, possible, testing retries
retries succesful
x150
18250, not enough models between
18200, possible, testing retries
retries succesful
x200
18300, not enough models between
18250, possible, testing retries
retries failed
18200, want to select the same starting model as last time. selecting earlier model.
18150, possible, testing retries
retries succesful
x150
18250, not enough models between
18200, possible, testing retries
retries succesful
x200
18300, not enough models between
18250, possible, testing retries
retries failed
18200, want to select the same starting model as last time. selecting earlier model.
18150, possible, testing retries
retries succesful
x150
18250, not enough models between
18200, possible, testing retries
retries succesful
x200
```

This is another example of a model that experiences a similar issue.

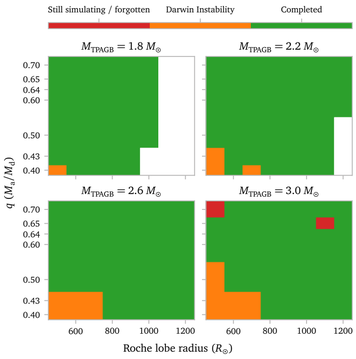
```
cleaning dir
making binaries
22750, not enough models between
22700, possible, testing retries
retries failed
22650, possible, testing retries
retries succesful
x650
22700, not enough models between
22650, want to select the same starting model as last time. selecting earlier model.
22600, possible, testing retries
retries succesful
x600
30050, not enough models between
30000, possible, testing retries
retries succesful
30000
30000, not enough models between
29950, possible, testing retries
retries succesful
x950
30050, possible, testing retries
retries failed
30000, possible, testing retries
retries succesful
30000
30100, not enough models between
30050, possible, testing retries
retries failed
30000, want to select the same starting model as last time. selecting earlier model.
29950, possible, testing retries
retries succesful
x950
30050, possible, testing retries
retries failed
30000, possible, testing retries
retries succesful
30000
30100, not enough models between
30050, possible, testing retries
retries failed
30000, want to select the same starting model as last time. selecting earlier model.
29950, possible, testing retries
retries succesful
x950
30050, possible, testing retries
retries failed
30000, possible, testing retries
retries succesful
30000
30100, not enough models between
30050, possible, testing retries
retries failed
30000, want to select the same starting model as last time. selecting earlier model.
29950, possible, testing retries
retries succesful
x950
30050, possible, testing retries
retries failed
30000, possible, testing retries
retries succesful
30000
30100, not enough models between
30050, possible, testing retries
retries failed
30000, want to select the same starting model as last time. selecting earlier model.
29950, possible, testing retries
retries succesful
x950
```

Luckily, we can fix this quite easily.

Below, I write the updated main loop:

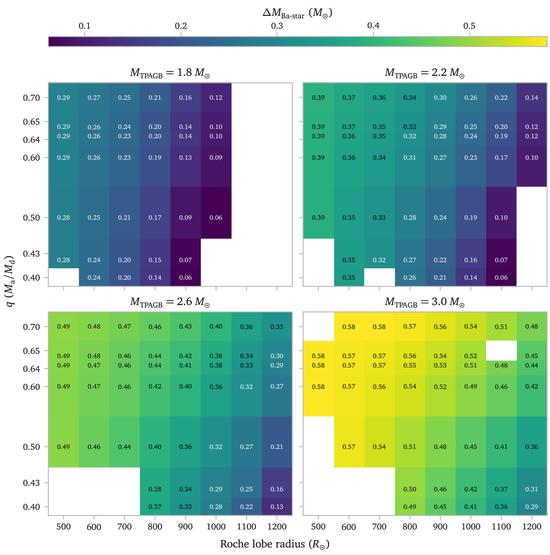
```python
# the beginning is the same

print("cleaning dir")
subprocess.run(["./clean"])
print("making binaries")
subprocess.run(["./mk"])

restarts = 1


try:
    history = mr.MesaData(f"LOGS/history.data")
except FileNotFoundError:
    print("running from scratch because data is missing")
    subprocess.run([f"./rn"])

# i introduce a boolean "strict", which is on by default.
# it chances the conditions a photo has to satisfy in order
# to be accepted as the starting photo.
# if strict:
#   -   difference between starting photo and final model
#       of the previous run has to be greater than 30 models.
#   -   starting photo and photo before it need to have the
#       exact same number of retries
#
# if not strict:
#   -   photo only needs to be older than the final model
#       of the last simulation.
#   -   starting photo can have 3 retries more than the
#       photo before it.

strict = True

# prev_start_models is now a list to keep all previous starts stored.
prev_start_models = []

# i increase the limit of retries to 15.
while restarts <= 15 and not check_finished():

    print(f"{prev_start_models = }")

    photo_int, mesh_delta_coeff, start_model = get_restart(prev_start_models, strict)
    update_inlist(mesh_delta_coeff)

    print(photo_int)
    subprocess.run([f"./re", photo_int])

    if check_finished():
        break

    # if the simulation moved past a troublesome part, we can remove the
    # previous photos and return to the strict mode if we are no longer
    # in this mode.

    if len(prev_start_models) > 0 and start_model > np.max(prev_start_models) + 149:
        prev_start_models = []
        strict = True

    # we add the previous photo to the list of starting models.

    prev_start_models.append(start_model)

    restarts += 1

    # if we encountered 3 restarts at a similiar point without making progress,
    # we disable strict mode and try again.

    if len(prev_start_models) > 3 and strict == True:
        prev_start_models = []
        strict = False
```

d.1 Results



After the second fix, nearly all models completed, with the two exceptions being:

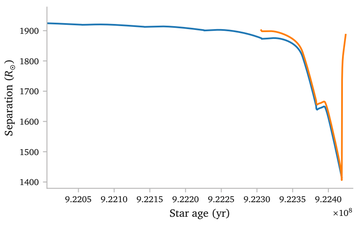
1. A model is still simulating and experiencing a late thermal pulse, which takes a long time
2. I forgot to simulate a model that did not strip its envelope with the new fix.



e. Problem with Grid Setup

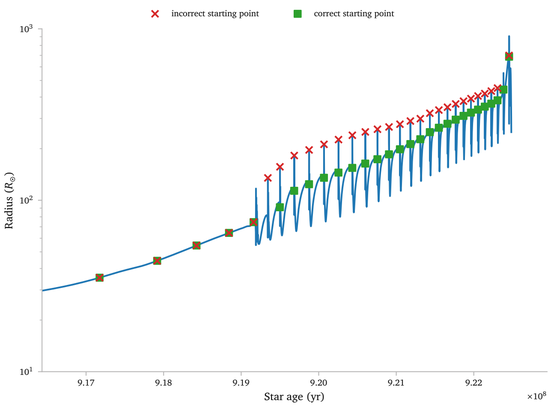
When working on some dredge-up calculations, I kept running into a problem where the simple binary evolution would not agree with the start of the detailed *MESA* binary evolution.

Here is an example:

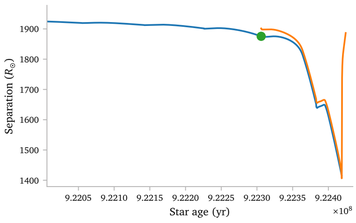


After some debugging, I was able to attribute this to the way I determine the starting conditions of the starting models. This was based on the values of the peak, instead of the values at the actual start[3].

3 because those are not the same.



I have now fixed this!



The green dot is the starting position of the binary *after* fixing the grid generation code.

f. SLURM change

When inspecting the progress of my *MESA* models, I saw that Karel was also running some models and he set the time limit to 1 days, 23 hours and 45 minutes instead of exactly 2 days. I suspect this is to give SLURM time to transfer the models after a job has been finished and I am going to use this in the future (even though I do not suspect my models will need to simulate for a full 2 days).

g. TODO

I want to implement some form of caching where I save an array of β-accretion values for the simple binaries i simulate without python. This would save a lot of time in the future for the TDU calculations where i need the accretion efficiency as a function of time.

References

Karakas, A. I. & Lugaro, M. (2016) *Stellar Yields from Metal-rich Asymptotic Giant Branch Models*. ApJ825, 26.

A Full Restart code

```
import shutil
import subprocess
import os
import re
import numpy as np
import mesa_reader as mr
import sys
from pathlib import Path

def get_mesh_delta_coeff(history, index):
    """
    exactly inverted logic compared to subroutine resolve_mesh_during_mass_loss(s, ierr)
    in run_dir/src/star/additional_routines.inc
    """

    tp_array = get_in_tp(history)
    in_TP = tp_array[-index]
    if in_TP:
        target_mesh_delta_coeff = 2.0

    elif (history.log_abs_mdot[-index]) > -5.5:
        target_mesh_delta_coeff = 1.0

    else:
        target_mesh_delta_coeff = 2.0

    return target_mesh_delta_coeff


def get_in_tp(history):
    """Return a boolean array indicating whether each history point is in a TP."""

    log_LHe = np.asarray(history.log_LHe)
    log_LH = np.asarray(history.log_LH)

    in_tp = np.zeros(len(log_LHe), dtype=bool)
    pulse_active = False

    for i in range(len(log_LHe)):

        if not pulse_active:
            # start thermal pulse
            if log_LHe[i] > 4:
                pulse_active = True

        else:
            # end thermal pulse
            if (log_LHe[i] < 3) and (log_LH[i] - log_LHe[i] > 1):
                pulse_active = False

        in_tp[i] = pulse_active

    return in_tp


def get_restart(prev_start_model):

    photos_path = f"photos/"
    history = mr.MesaData(f"LOGS/history.data")

    index = find_index(history, prev_start_model, verbose=1)

    final_restart_value = history.model_number[-index]
    if final_restart_value % 1000 == 0:
        photo_str = f"{int(final_restart_value)}"

    else:
        photo_str = f"x{int(str(final_restart_value)[-3:])}"

    return photo_str, get_mesh_delta_coeff(history, index), final_restart_value


def check_finished():
    history = mr.MesaData(f"LOGS/history.data")
    if history.period_days[-1] < 50:
        # model in inspiral, no need to continue
        return True

    final_model = Path("post_AGB.mod")
    return final_model.exists()


def find_index(history, prev_start_model, verbose=0):
    """
    conditions for restart:
        1. the model restarts at least 30 model steps before its initial termination
        2. the restart model has a log_dt that is larger than -3.5
        3. the restart model has the same number of retries as the last model saved
        ↪  before
        4. the restart model has a DIFFERENT model number than the previous restart
         ↪  model.
    """

    for i in range(1, len(history.model_number) + 1):

        if history.model_number[-1] - history.model_number[-i] >= 1000:
            mod = 1000
        else:
            mod = 50

        if history.model_number[-i] % mod == 0:
            if verbose:
                print(history.model_number[-i], end=", ")

            if history.model_number[-1] - history.model_number[-i] < 30:
                if verbose:
                    print("not enough models between")
                prev_possible = False
                prev_retries = None
                prev_index = None
                continue

            if history.log_dt[-i] < -3.5:
                if verbose:
                    print("log_dt too small")
                prev_possible = False
                prev_retries = None
                prev_index = None
                continue

            if history.model_number[-i] == prev_start_model:
                if verbose:
                    print(
                        "want to select the same starting model as last time. selecting
                        ↪  earlier model."
                    )
                prev_possible = False
                prev_retries = None
                prev_index = None
                continue

            if verbose:
                print("possible, testing retries")
            if history.num_retries[-i] == history.num_retries[-i - 5]:
                print("retries succesful")
                return i
            if verbose:
                print("retries failed")
    raise Exception("No restart model found")


def update_inlist(mesh_delta_coeff):
    """
    this function updates the inlists for the retries. it does 2 things.

    1: it ADDS the line if it is not present:
        min_timestep_limit = 30d0
    because some of my earlier binary models did not have it, and it is
    important not to let the binary models struggle too long once they
    encounter convergence problems.

    2: it changes the delta_mesh_coeff value either to 1, or to 2
    depending on the conditions of the retry model. it changes the mesh_delta_coeff
    value to the opposite value as the one we want to simulate the binary evolution with,
    such that we temporarily use a different mesh refinement and thus hopefully evolve
     ↪  past
    the instability.
    """
    with open("inlist_common", "r") as f:
        lines = f.readlines()
        new = []

        encountered_time_limit = False
        encountered_mesh_delta_coeff = False
        for line in lines:
            if "min_timestep_limit" in line:
                new.append(f"\tmin_timestep_limit = 30d0\n")
                encountered_time_limit = True
            elif "mesh_delta_coeff" in line:
                new.append(f"\tmesh_delta_coeff = {mesh_delta_coeff}d0\n")
                encountered_mesh_delta_coeff = True
            else:
                new.append(line)

        if not encountered_time_limit:
            new.insert(-2, "\tmin_timestep_limit = 30d0\n")

        if not encountered_mesh_delta_coeff:
            new.insert(-2, f"\tmesh_delta_coeff = {mesh_delta_coeff}d0\n")

    with open("inlist_common", "w") as f:
        f.writelines(new)


print("cleaning dir")
subprocess.run(["./clean"])
print("making binaries")
subprocess.run(["./mk"])

restarts = 1


try:
    history = mr.MesaData(f"LOGS/history.data")
except FileNotFoundError:
    print("running from scratch because data is missing")
    subprocess.run([f"./rn"])


prev_start_model = None
while restarts <= 10 and not check_finished():

    photo_int, mesh_delta_coeff, start_model = get_restart(prev_start_model)
    update_inlist(mesh_delta_coeff)

    print(photo_int)
    subprocess.run([f"./re", photo_int])

    if check_finished():
        break
    prev_start_model = start_model

    restarts += 1
```